

Introducción a la Computación Gráfica

Obligatorio 1
Curso: 2025

Franco Campanella	5.329.377-7
Lucas Barrientos	5.453.900-9

Índice

Índice	1
Introducción	3
Definición del problema	3
Análisis del problema	3
Diseño de la solución	4
Arquitectura	5
Implementación	7
Descripción del entorno utilizado	7
Algoritmos y técnicas gráficas implementadas	7
SDL	8
SDL_image	8
SDL_ttf	8
SDL_mixer	8
glm	8
assimp	8
pugixml	8
Código desarrollado por terceros	9
Estados de la aplicación	10
MainMenu	10
GamePlay	11
SettingsMenu	11
LevelCreator	12
Grilla tridimensional	13
Lista encadenada	13
Vectores	13
Velocidad de simulación	13
Uso de OpenGL	13
Manejo del estado de OpenGL	14
Iluminación, color y materiales	14
Display List	14
Vertex arrays	14
Texturas	15
Modelos 3D	15
Cámara	16
HUD	17
Desarrollo del obligatorio	18
Cargado de información	18
Niveles	18

Niveles personalizados	18
Archivo de guardado	19
Movimiento	20
Colisiones	20
Settings	21
Características particulares de la solución	23
Sonidos	23
Sistema de partículas	23
Diseño de niveles	23
Algunos desafíos superados	23
Conclusiones	26
Trabajo futuro	27
Referencias	28

Introducción

Definición del problema

Para esta tarea se nos planteó el desarrollar una versión en 3D del videojuego Apple Worm, utilizando como herramientas principales las librerías OpenGL, GLU y SDL. El objetivo de crear este videojuego es el plasmar las diferentes posibilidades que permite OpenGL (iluminación, diferentes tipos de vistas y perspectivas, animaciones, etc) a través de distintos algoritmos que llevan a cabo el dibujado de píxeles, mismos algoritmos que han sido explorados en el curso de Computación Gráfica.

Análisis del problema

Se nos plantea el problema de adaptar un juego de puzzles en 2d al 3d. Se nos dan las opciones de basarnos en el juego y simplemente agregar los gráficos en 3d, o de adaptar la jugabilidad a un mundo 3d.

Decidimos ir con la segunda opción, ya que permitiría diseño de niveles más interesantes, desde esta base comenzamos a hacer algunos dibujos y diagramas de cómo pensamos que el juego se iba a ver, y detectamos los principales puntos en los que el desarrollo se tendría que centrar en un comienzo:

1. Encontrar una estructura de datos que almacene los niveles y encontrar una forma de que esta pudiera tener cualquier tipo de entidades que queramos agregar.
2. Pensar en cómo funciona el movimiento de la viborita, y cómo lo íbamos a implementar
3. Pensar en cómo vamos a implementar la interfaz gráfica utilizando las herramientas disponibles: OpenGL y SDL2.

Estos 3 puntos son fundacionales para el juego, y una vez solucionados el resultado sería un juego en un estado casi entregable que incluiría:

- Niveles variables
- Jugabilidad
- Interfaz

Por lo que desde esta base comenzamos el desarrollo, con el objetivo de completar estos 3 puntos en la primera semana de desarrollo.

Luego de completar esta primera etapa, intentaremos seguir con la implementación de todo lo obligatorio, por lo que tendríamos que comenzar a investigar la solución a estos problemas:

- Cómo cargar texturas
- Cómo cargar modelos
- Cómo implementar la iluminación

Diseño de la solución

En esta sección veamos cómo nos enfrentamos a los 3 primeros desafíos, y cómo siguió el desarrollo una vez estos fueron completados.

Primero, con respecto al mapa de un nivel, decidimos utilizar una grilla cúbica, en un principio elegimos un tamaño de 8x8x8, que planeábamos hacer dinámico, pero al final decidimos quedarnos con estas dimensiones. Cada posición en la grilla puede ser NULL (vacía) o puede contener un IGameEntity, esta interfaz indica que el objeto es una entidad del juego, con la que el jugador va a poder interactuar, las entidades iniciales en el proyecto fueron:

- Block: el cubo sólido con el que construir los niveles
- Viborita: la viborita controlada por el jugador
- Apple: las manzanas que determinan el puntaje del jugador y que dan tamaño a la viborita
- Goal: la meta, que al tocar la viborita con la cabeza, completa el nivel.

Más adelante se agregó la SpikedApple, como adaptación de la mecánica de pinchos del juego original, decidimos hacer manzanas trampa en lugar de pinchos porque nos daba la flexibilidad de colocarlas en cualquier lugar del nivel (ya que flotan) y además no nos teníamos que preocupar de rotaciones en 3 dimensiones del modelo de pinchos para colocarlo en el mapa.

En cuanto al movimiento del gusano se refiere, decidimos que fuese a través de los ejes X, Y y Z, utilizando las teclas UP, DOWN, RIGHT, LEFT en los ejes X y Z, y, las teclas Z y X en el eje Y. Para la implementación, utilizamos una lista encadenada, con un header que tuviera referencias al primer y último elemento, en la función handleMovement de la viborita, se recibe una dirección, se chequea por colisiones con otros elementos de la grilla, si se puede mover en esa dirección entonces mueve la cabeza. Luego de esto recorre el resto del cuerpo moviendo cada parte a la posición previa de la parte anterior.

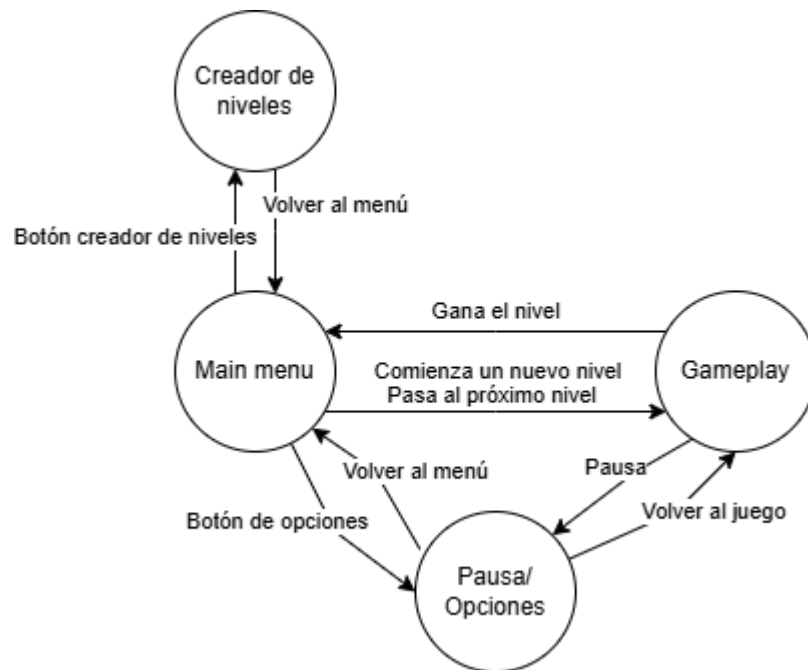
Finalmente, para la implementación de la interfaz se definió que todo sería por medio de dibujar rectángulos con imágenes como texturas, usamos la estructura SDL_Rect para almacenar las coordenadas y tamaño de los rectángulos, ya que esta nos daba la función SDL_HasIntersection, que nos permite chequear si se superponen dos SDL_Rects, utilizando esto creamos un rectángulo de 1x1 en la posición del mouse, y de esta forma conseguimos una usabilidad de la interfaz con el mouse.

Una vez terminada esta etapa inicial del diseño, continuamos con la inclusión de modelos, para esto utilizamos Blender 3D como herramienta para modelar, y posteriormente, cargamos los modelos utilizando la librería Assimp [8].

Para crear los distintos niveles, se integró la librería pugixml, definiendo un formato xml con el que poder agregar niveles. Esta forma de trabajar rápidamente se volvió poco utilizable con niveles medianamente complejos, por lo que implementamos un creador de niveles el cual permite crear un nivel utilizando las entidades del juego desde una interfaz gráfica, que además nos permite testear el nivel a medida que lo desarrollamos.

Arquitectura

Para la arquitectura comenzamos definiendo esta máquina de estados en los que puede estar el juego:

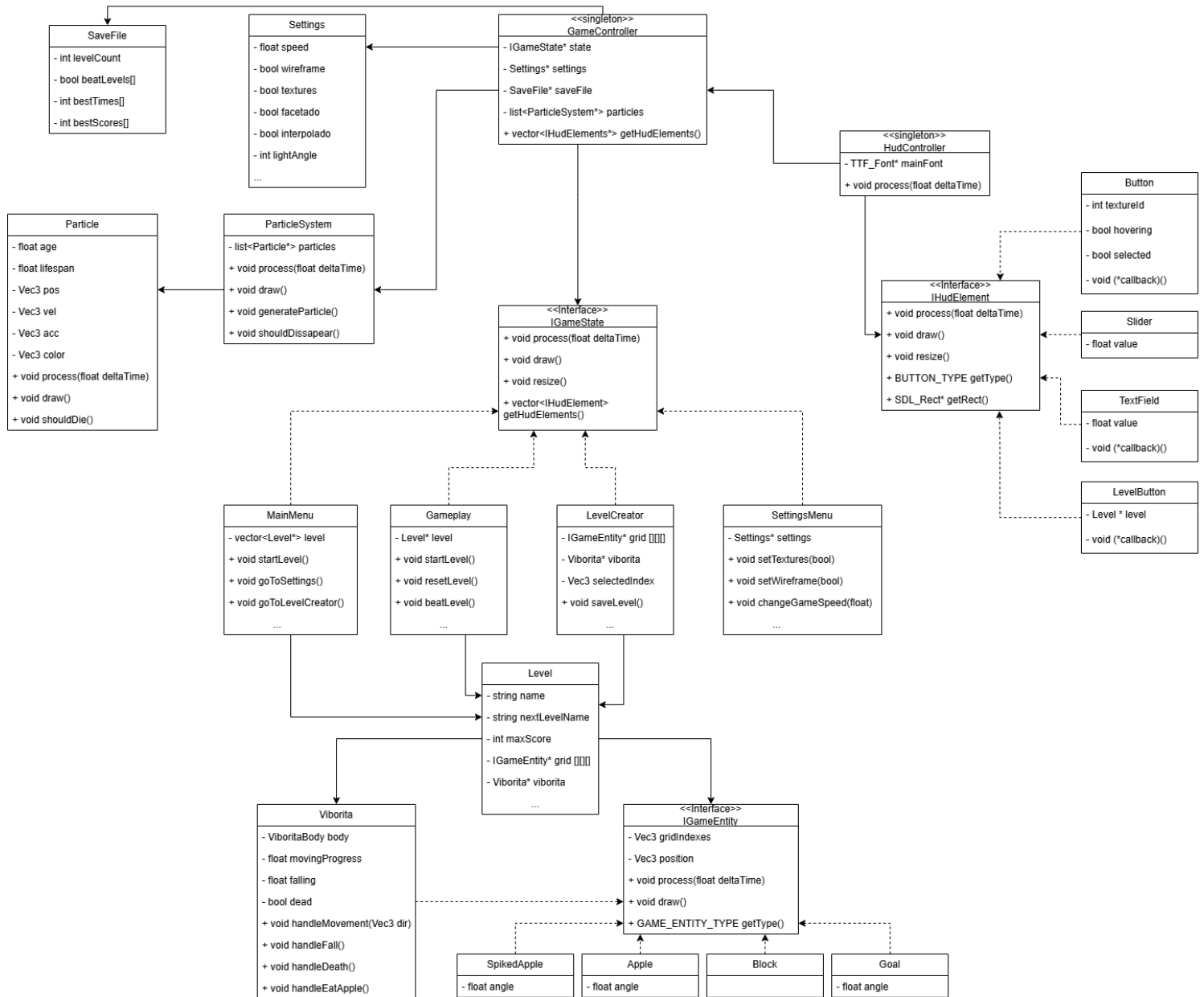


Estos estados definen qué se renderiza en la pantalla, tanto los elementos 3d del juego, como la interfaz proyectada en 2d.

La clase encargada de manejar estos estados es el GameController, este cuenta con la función processFrame, llamada cada frame, que se encarga de llamar al process del estado actual, también es el encargado de:

- Dibujar partículas
- Aplicar configuración de wireframe
- Llamar al HudController para que dibuje el hud

Este es un uml simplificado (sin todas las propiedades y métodos) de las clases usadas en el juego:



Como vemos la mayoría de elementos que dibujan algo en pantalla tienen una función `process(float deltaTime)` y una función `draw()`, esto permite una fácil separación del procesado (progreso de animaciones, lectura de inputs, etc), y lo que dibuja en pantalla ese elemento. Esta estructura está parcialmente inspirada en el motor de juegos Godot.

Otro archivo importante que usamos para varias cosas es el `utils.cpp` y `utils.h`, estos contienen varias variables globales por defecto (como los vértices para un cubo), tiene la definición de los enum que usamos en el código. También cuenta con funciones generales que utilizamos desde varios lugares, algunas de estas son: `loadTexture`, `cargarModelo`, `drawModel`, `drawCubeWithNormals`, `playSound` y muchas más.

Implementación

Descripción del entorno utilizado

El desarrollo se llevó a cabo utilizando Visual Studio 2022, con el compilador por defecto de esta MSVC 14.3. Se desarrolló con y para Windows, más específicamente se desarrolló en Windows 10.

Algunas herramientas que utilizamos tanto para asistir con el desarrollo, como para otras actividades necesarias fueron:

- Blender, al decidir hacer los modelos 3d nosotros mismos, esta es la herramienta open source por excelencia para el modelado, por lo que la utilizamos para los modelos de:
 - La viborita
 - Las manzanas
 - Las manzanas con pinchos
 - La meta
 - El mapa alrededor del juego [1], este modelo no fue realizado por nosotros, y solo fue modificado parcialmente para incluirlo en el juego, se modificaron los colores y se eliminó la esfera general que contenía el modelo.
- Inkscape, esta es una herramienta open source para el diseño de gráficos vectoriales, esta se utilizó en el contexto de la tarea para diseñar las texturas del bloque, las texturas de las flechas en la interfaz del Gameplay, los tutoriales de los primeros niveles.
- Gimp, esta es una herramienta open source para la edición de imágenes, se utilizó para la edición de algunas imágenes usadas en botones.

Algoritmos y técnicas gráficas implementadas

Sombreado de malla poligonal

Este algoritmo fue utilizado como una forma de ver a los modelos cuando una luz incide sobre ellos, este es uno de los algoritmos de sombreado implementados, se usa utilizando la función `glShadeMode` con el modo `GL_FLAT`, el cual asigna un solo color a toda la superficie del polígono. Si bien nos brinda una aproximación de cómo incide la luz en los objetos, no es muy realista ya que podemos ver el sombreado entre los diferentes vértices que forman el modelo, esto es debido al efecto de las bandas de Match; como alternativa el jugador puede cambiarlo en las settings por un sombreado de Gouraud.

Sombreado de Gouraud

Este algoritmo, a diferencia del anterior, podemos ver como la superficie del objeto en donde incide la luz se ve de un tono más suave y sin el efecto de las bandas de Match, para esto se utiliza la función `glShadeMode` con el modo `GL_SMOOTH`, el color interior al polígono es el resultado de interpolar el color de sus vértices.

Librerías utilizadas

SDL

Esta es una de las librerías dadas en el curso, se utilizó para:

- La creación de la ventana
- El manejo de eventos, esto incluye escuchar inputs del teclado y mouse, también incluye inputs de resize de la ventana.
- Utilizamos la estructura `SDL_Rect` para facilitar la implementación del hud

SDL_image

Librería para la carga de imágenes, esta librería genera una superficie en base a una imagen que podemos asignar a una textura, de esta forma podemos asignar imágenes a botones o a modelos.

SDL_ttf

Librería para el uso de fuentes, nos permitió importar una fuente personalizada y, de nuevo, generar superficies con el texto que queramos, que a su vez asignamos a una textura que luego podemos utilizar para elementos de la interfaz.

SDL_mixer

Librería que facilita la carga y reproducción de imágenes, SDL2 ya cuenta con soporte para reproducir audios, pero es muy elemental y requiere mayor trabajo por parte del desarrollador, en nuestro caso como solo necesitábamos reproducir audios precargados, `SDL_Mixer` nos dio funciones simples que permiten:

- Cargar audios
- Reproducir los audios
- Modificar el volumen de los audios

glm

assimp

Librería utilizada para cargar modelos 3D. Permite extraer la información importante de los modelos como vértices, normales, mallas, etc, información utilizada para el dibujado de objetos.

pugixml

Esta librería la usamos para manipular archivos XML, los archivos correspondientes a los niveles del juego son de este tipo. Sirve para leer, escribir y modificar documentos XML. Gracias a esto logramos crear y posteriormente guardar los niveles.

Código desarrollado por terceros

Utilizamos chat GPT como ayuda para escribir algunas funciones específicas del programa, estas fueron:

- cargarModelo
- drawModel
- getRandomInt
- getRandomFloat
- setSaveFile (solo la lectura del archivo)
- saveSaveFile (solo el guardar el archivo)

También nos ayudó a generar ciertos arreglos de datos básicos, como los arreglos de vértices, índices y normales del cubo base.

El código para el sistema de partículas está basado en un tutorial [4], sin embargo los parámetros y el contexto en el que se usó fue adaptado para nuestras necesidades.

Estados de la aplicación

MainMenu



El menú principal es la primera pantalla que el usuario verá, consiste de una mezcla de elementos 3d y 2d para la interfaz.

En el 3d vemos el mundo de decoración alrededor, vemos el título del juego escrito con varias viboritas, y vemos varias animaciones del juego mostrándonos algunos niveles y un poco de la jugabilidad.

En el 2d vemos las opciones básicas que necesitamos para ir al resto de estados de la aplicación, las opciones de Niveles y Niveles personalizados nos llevan a carruseles en los que elegir el nivel que vamos a jugar:



- Presionar uno de estos niveles nos lleva al estado de Gameplay
- El botón de Creador de niveles nos lleva al estado LevelCreator.
- El botón de Settings nos lleva al estado SettingsMenu.
- Salir cierra el juego.

GamePlay



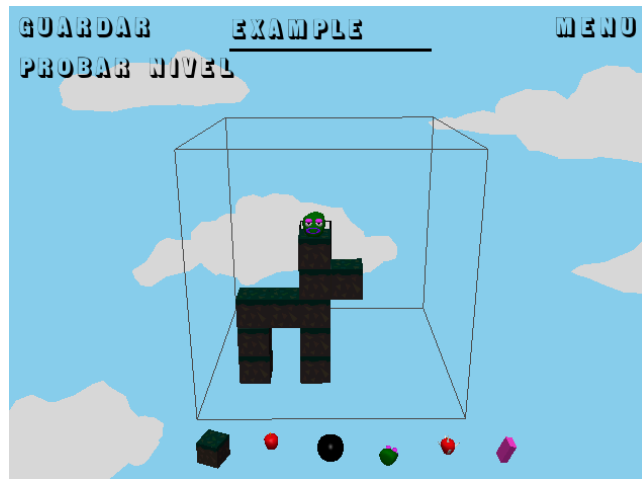
Este es el estado en el que jugamos al juego, vemos una interfaz con los elementos básicos pedidos y algún elemento extra, como las flechas que indican la orientación de movimiento en relación a la cámara, y dependiendo del nivel una ilustración tutorial como pista de cómo completar el nivel.

SettingsMenu



Este es el menú en el que configuramos los parámetros pedidos, podemos entrar en dos contextos distintos, como pausa mientras jugamos, o desde el menú. Si entramos mientras jugamos un nivel seguiremos viendo el nivel pausado y podremos rotar la cámara y ver en tiempo real cómo impactan las configuraciones que hagamos. Si entramos desde el menú no veremos ningún nivel y solo podremos modificar las opciones.

LevelCreator



Esta pantalla nos permite crear niveles personalizados y guardarlos para jugar posteriormente.

Cuenta con una interfaz con varios elementos, estos son:

- Botón de guardar, guarda el nivel en un archivo .xml en la carpeta de customLevel, solo podemos guardar un nivel una vez haya una viborita colocada.
- Botón de probar nivel, nos permite jugar el nivel para asegurarnos de que sea posible de completar o simplemente testearlo
- Campo de nombre, nos permite colocar un nombre de hasta 11 caracteres
- Botón de menú, hace que volvamos al menú, esto borra el nivel que estábamos creando si no lo guardamos previamente.
- Barra de entidades, esta nos permite seleccionar qué entidad colocar en el mapa, la entidad seleccionada gira para indicar su estado. El último modelo que vemos es la goma de borrar, que nos permite eliminar entidades del nivel.

Los controles del creador de niveles son los mismos que para mover la viborita, pero se agrega la barra espaciadora para colocar entidades.

Estructuras de datos

Grilla tridimensional

Utilizamos una matriz de 8x8x8 para almacenar el contenido de los niveles, esta contiene punteros a IGameEntity, que pueden ser cualquiera de los elementos del juego.

Lista encadenada

La lista encadenada fue utilizada para representar el cuerpo de la serpiente, además tenemos un puntero hacia la primera y última posición de la lista, posiciones que representan la cabeza y cola de la viborita, cada vez que la viborita come una manzana, se agrega un nuevo elemento en la posición previa al movimiento de la cola y se suma en 1 su tamaño, cada parte de su cuerpo es un struct ViboritaPart, el cual contiene un puntero a la siguiente parte, y , un Vec2 lastRotation que servirá para saber a donde rotar el cuerpo al agregar animaciones. También contiene 6 Vec3 (struct que contiene las coordenadas x, y, z) los cuales corresponden a dirToFront, orientationToFront, gridIndex, position, lastRotation, lastOffset, lastMovement, los primeros 4 sirven para saber hacia dónde se mueve la viborita y en donde se encuentra, además, toda esta información será utilizada para saber hacia donde mover y rotar el cuerpo de la viborita, para esto son claves las funciones calculatePartRotation y translateAndRotateBodyPart que serán usadas en la función draw de la viborita lo cual dará un efecto de animación de movimiento por el mapa.

Vectores

Utilizamos vectores para almacenar distintos objetos, por ejemplo IHudElements.

Velocidad de simulación

Para que el movimiento y usabilidad fuera la misma sin importar las especificaciones del sistema utilizamos una variable llamada deltaTime, esta es comúnmente usada en el desarrollo de videojuegos para que las físicas no se vean afectadas por las imágenes por segundo que tenga el usuario.

En nuestro caso utilizamos una función que nos da SDL para obtener el tick actual, luego hacemos la diferencia con el tick anterior y lo dividimos entre la frecuencia de ticks, esto nos da un resultado en segundos del tiempo entre imágenes.

Siempre que algo se mueve o tiene animaciones continuas se multiplica su velocidad por deltaTime, el uso de esta variable además nos permitió implementar la configuración de velocidad fácilmente, con solo multiplicar deltaTime por la velocidad que queremos que vaya el juego todo va más rápido o más lento en función de la configuración.

Uso de OpenGL

Antes de inicializar variables creamos la escena, para lograrlo creamos una ventana de dimensiones 640x480 y agregamos un contexto asociado a esta ventana para realizar renderizado gráfico, luego empezamos a configurar la escena, primero fijamos el color por defecto que tendría la imagen antes de renderizar objetos usando glClearColor, esta función se utiliza para establecer el color que se utilizará al borrar el búfer de color. Luego, se hizo uso de la función gluPerspective para generar un frustum en el sistema de coordenadas del mundo, ajustamos el campo de visión a 45°, hicimos que la relación de aspecto coincidiera con las dimensiones de nuestra ventana y definimos el zNear y el zFar en 0.1 y 100 respectivamente. Ahora, para ver la profundidad de la escena usamos la función glEnable para habilitar opciones, con la bandera GL_DEPTH_TEST. Por último, ejecutamos glMatrixMode para establecer el modo de matriz actual, en otras palabras, cuál de las pilas disponibles de OpenGL se usará para aplicar las diferentes transformaciones que ofrece la librería.

Manejo del estado de OpenGL

Hay una serie de banderas que activamos y desactivamos cada frame según necesitemos, estas son:

- GL_LIGHTING, se activa al renderizar los niveles y los elementos 3d que lo requieran, luego se desactiva. Cada estado del juego (IGameState) maneja esta flag
- GL_COLOR_MATERIAL, se deja activado para el correcto renderizado de los colores cuando se aplica un material al modelo.
- GL_NORMALIZE, se deja activado para poder utilizar el arreglo de normales y que la luz se renderice correctamente
- GL_BLEND, se utiliza para renderizar elementos de hud y que aparezcan con fondos transparentes al estar sobre elementos 3d.
- GL_TEXTURE_2D, utilizado para aplicar texturas a los polígonos que se renderizan.

Para manejar los distintos cambios de opciones creamos la clase Settings, la cual se encarga de contener las variables que almacenan la información de los distintos parámetros, además, a través de esta clase es que podemos modificar los valores de estas variables, entonces, si el jugador quiere cambiar alguna opción, el SettingsMenu se encarga de tomar estos nuevos valores y modificar su atributo settings con estos, hecho esto, los datos en pantalla serán renderizados en base a los nuevos valores introducidos.

Iluminación, color y materiales

Para el uso de materiales aplicados en los modelos, creamos los materiales a partir de un struct Material, los elementos son arreglos que contienen las características de cada luz, las cuales refieren a cómo actúan sobre ellos los diferentes tipos de luces que existen (ambiente, especular y difusa), y, el brillo. Al aplicar estos materiales sobre los modelos veríamos cómo impacta la iluminación de la escena sobre ellos. Para la iluminación, creamos un struct Light, en el que, al igual que en Material, define las características de cada luz de la escena, a partir de esto creamos un foco de luz el cual actúa de “sol”, al cambiar su posición (esto se hace en las settings) podemos ver cómo varía la iluminación de la escena. Para los colores de los modelos simplemente utilizamos la función glColor3f que brinda OpenGL, aplicando el color cargado del .mtl de cada modelo.

Display List

No utilizamos displayLists, esto podría ser una potencial mejora para la performance del juego (aunque actualmente no la necesita), podríamos haber utilizado displayLists para el renderizado de los modelos, ya que en los niveles siempre renderizamos los mismos modelos varias veces sin cambio, solo aplicando distintas transformaciones a la matriz ModelView.

Vertex arrays

Utilizamos vertex arrays para el renderizado de todos los modelos 3d del juego, esto incluye los que realizamos en blender, y el cubo que renderizamos como bloque.

Para los modelos de blender almacenamos el VertexArray en el struct de mesh que tenemos y luego lo preparamos para dibujar con la línea glVertexPointer, luego con glDrawElements dibujamos el modelo con los vértices definidos.

Además de Vertex array utilizamos un arreglo de índices, un arreglo de coordenadas de texturas y un arreglo de normales.

Texturas

Para el manejo de texturas creamos la función `loadTexture`, esta se encarga de cargar una imagen utilizando la librería `SDL_image`, esta imagen se obtiene como una `surface`, ahora, llamamos a `SDL_ConvertSurfaceFormat` para pasar nuestro `surface` al formato de `RGBA`, para esto utilizamos el enumerado `SDL_PIXELFORMAT_RGBA32`. Una vez hecho esto generamos un `id` de textura y lo enlazamos a la textura ya cargada, para esto usamos `glGenTextures` y `glBindTexture`, por último, configuramos las características de nuestra textura con `glTexParameteri`.

Esta `id` de textura queda en `map` de `ids` de texturas con el `path` como clave. Esto nos permite no cargar la misma textura varias veces cuando la pidamos.

La función retorna el valor de la `id` generada u obtenida del `map` por medio de un parámetro por referencia.

Hay otra función muy similar a esta que se encarga de generar texturas con texto, esta es `loadTextTexture`. Utiliza la librería `SDL_ttf` para generar la superficie. Esta función no guarda la `id` en el `hash`, ya que las texturas de texto se limpian cada vez que se destruye el botón que las use.

Modelos 3D

Para cargar los modelos 3D en la aplicación utilizamos la función `cargarModelo`, la cual recibe el `path` del archivo, el nombre y la posición a ocupar en el vector de modelos cargados. Lo que hace esta función es procesar cada malla del modelo, por cada malla se guarda la posición, la normal, las coordenadas de la textura y el color de cada vértice, además, se procesan las caras de la malla, una vez procesada cada malla del modelo, este es guardado en el vector `modelsInfo` que contiene la información de cada modelo. Cada posición del `modelsInfo` contiene un nombre, un `GLuint` que corresponde a la `Id` de la textura (si es que tiene) y un vector de `subMesh`; `subMesh` es un `struct` que contiene un vector de vértices, un vector de normales y un vector de índices. Una vez cargado un modelo, llamamos a la función `drawModel` si queremos dibujarlo en pantalla, a esta se le pasa el tipo de modelo a dibujar y las texturas (si es que tiene), lo que hace esta función es dibujar cada malla con sus respectivos colores, haciendo uso de la función `glDrawElements` de `OpenGL`.

Modelos utilizados:

- Cabeza de la viborita: representa al primer elemento de la lista encadenada que compone a la viborita.
- Cabeza de la viborita muerta: representa al primer elemento de la lista encadenada que compone a la viborita si esta muere.
- Cuerpo de la viborita: representa a los elementos intermedios de la lista encadenada que compone a la viborita. Cada vez que la viborita come una manzana se agrega un nuevo elemento a la lista, por lo tanto, tenemos una nueva parte del cuerpo.
- Cola de la viborita: representa al último elemento de la lista encadenada que compone a la viborita.
- Manzana: están repartidas en el mapa y pueden ser devoradas por la viborita.
- Manzana con pinchos: están repartidas en el mapa y si la viborita las toca muere.
- Meta: puede encontrarse en cualquier posición del mapa, si la viborita se desplaza a su posición, ganas el juego.
- Título: se muestra como el título del juego en el menú principal.
- Borrador: es el modelo de la opción “Borrar” en el creador de niveles.

Cámara

El juego cuenta con 3 modos de cámara distintos:

La cámara principal del juego se mueve en una esfera alrededor de la grilla donde se renderiza el nivel, podemos movernos alrededor del nivel libremente en los límites de esta esfera.

Para implementarla utilizamos coordenadas polares, siempre que se mueve el mouse cambiamos los ángulos y con estos aplicamos las ecuaciones polares para calcular la posición nueva de la cámara, esta siempre apunta al punto (0,0,0).

La cámara secundaria es sobre el hombro, esta sigue a la viborita desde atrás y siempre apunta hacia donde mira la viborita en los ejes X y Z, si la viborita mira hacia arriba esta sigue en el plano X y Z.

Para implementarla se utiliza como centro un punto sobre su cabeza, y la posición se calcula con la ecuación de un círculo (en función al ángulo) alrededor de este, el ángulo se calcula de acuerdo de la orientación de la cabeza en el eje X y Z. Si la viborita se mueve se interpolan las posiciones y el ángulo de la cámara para que el movimiento sea continuo.

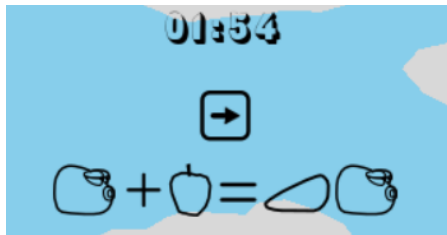
La última cámara es el modo libre, esta nos da libertad de movimiento por toda la escena, se controla con las teclas WASD y la rotación presionando un botón en el mouse y moviéndolo.

La implementación utiliza coordenadas polares nuevamente, para calcular hacia donde mira la cámara, el movimiento con WASD siempre es relativo al ángulo por lo que se utilizan 2 productos entre vectores para obtener una base normalizada rotada en base al ángulo, y cada tecla se mueve de acuerdo a esta base, con W nos movemos hacia donde mira la cámara, A y D nos mueven hacia la izquierda y derecha respectivamente, y la S nos mueve hacia atrás.

HUD

Hay 2 partes del hud:

El texto, botones, sliders y textfields son renderizados pasando a una perspectiva ortogonal que renderiza rectángulos con texturas en 2 dimensiones, algunos ejemplos de la interfaz en 2d:



Timer y tutoriales



Sliders



Textfield con placeholder

Luego hay elementos 3d que renderizamos como parte de la interfaz, para esto utilizamos una función que calcula una posición del elemento de tal forma que podemos elegir que tan arriba o a la derecha está en la pantalla, esta función es `calc3dCoordsForHud`, esta recibe la cámara actual, posición y centro, y usando el producto cruz calcula una base de vectores normalizada angulada como la cámara, esto permite agregar offsets de acuerdo a esta base y colocar el elemento 3d en la misma posición en la pantalla todos los frames, esto se usa para los siguientes elementos:



Estos elementos son:

- Título
- Barra de seleccion en el creador de niveles
- Preview de niveles
- Flechas orientadoras

Desarrollo del obligatorio

Cargado de información

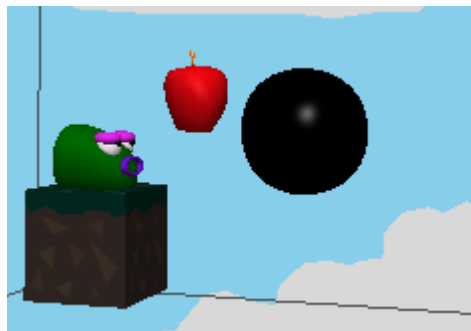
Niveles

Los niveles se guardan en .xml, que pueden luego ser leídos por la librería pugixml [3]. El formato fue definido por nosotros, este es un ejemplo de un nivel simple:

El xml:

```
<?xml version="1.0"?>
<Level name="ejemplo" maxScore="1" nextLevel="">
  <Grid>
    <GameEntity type="goal" x="2" y="2" z="2" />
    <GameEntity type="apple" x="3" y="2" z="2" />
    <GameEntity type="block" x="4" y="1" z="2" />
  </Grid>
  <Viborita>
    <BodyPart x="4" y="2" z="2" />
  </Viborita>
</Level>
```

El nivel:



Este formato nos permite colocar cada elemento en los índices de la grilla que queramos, y además nos permite tener una viborita del tamaño que queramos al iniciar el nivel. También nos permite colocar el nombre del próximo nivel, para tener una transición directa sin pasar por el menú principal.

Los niveles se guardan en la carpeta levels, desde el root del juego.

Niveles personalizados

Tienen el mismo formato y funcionalidad que los niveles, pero se guardan en la carpeta customLevels, esto es para separar los niveles que cree el usuario de los niveles hechos para el juego principal.

Archivo de guardado

El archivo de guardado es save.snaked, se guarda en un formato similar a un csv, este tiene 3 filas y n columnas, siendo n la cantidad de niveles principales del juego. Cada fila guarda información diferente: la primera fila guarda un boolean para saber si el nivel fue completado o no,

la segunda fila guarda el mejor tiempo del jugador en ese nivel, la tercera fila guarda el mejor puntaje del jugador en el nivel.

Este archivo se carga en el juego luego de la carga de niveles, ya que se necesita saber cuántos niveles hay en total.

Nos permite tener persistencia del progreso del jugador en los niveles principales.

Movimiento

El movimiento de la viborita siempre es discreto en una dirección, para esto generamos un vector de largo 3 normalizado, que tiene valores -1 0 o 1. Esto nos da 6 vectores posibles que son las 6 direcciones posibles en 3 dimensiones:

- Flecha derecha: movimiento hacia la derecha (eje X, (-1,0,0)).
- Flecha Izquierda: movimiento hacia la izquierda (eje X (1,0,0)).
- Flecha arriba: movimiento hacia la derecha (eje Z (0,0,1)).
- Flecha abajo: movimiento hacia la izquierda (eje Z (0,0,-1)).
- Tecla Z: movimiento hacia arriba. (0,1,0)
- Tecla X: movimiento hacia abajo. (0,-1,0)

Para las animaciones utilizamos principalmente dos técnicas:

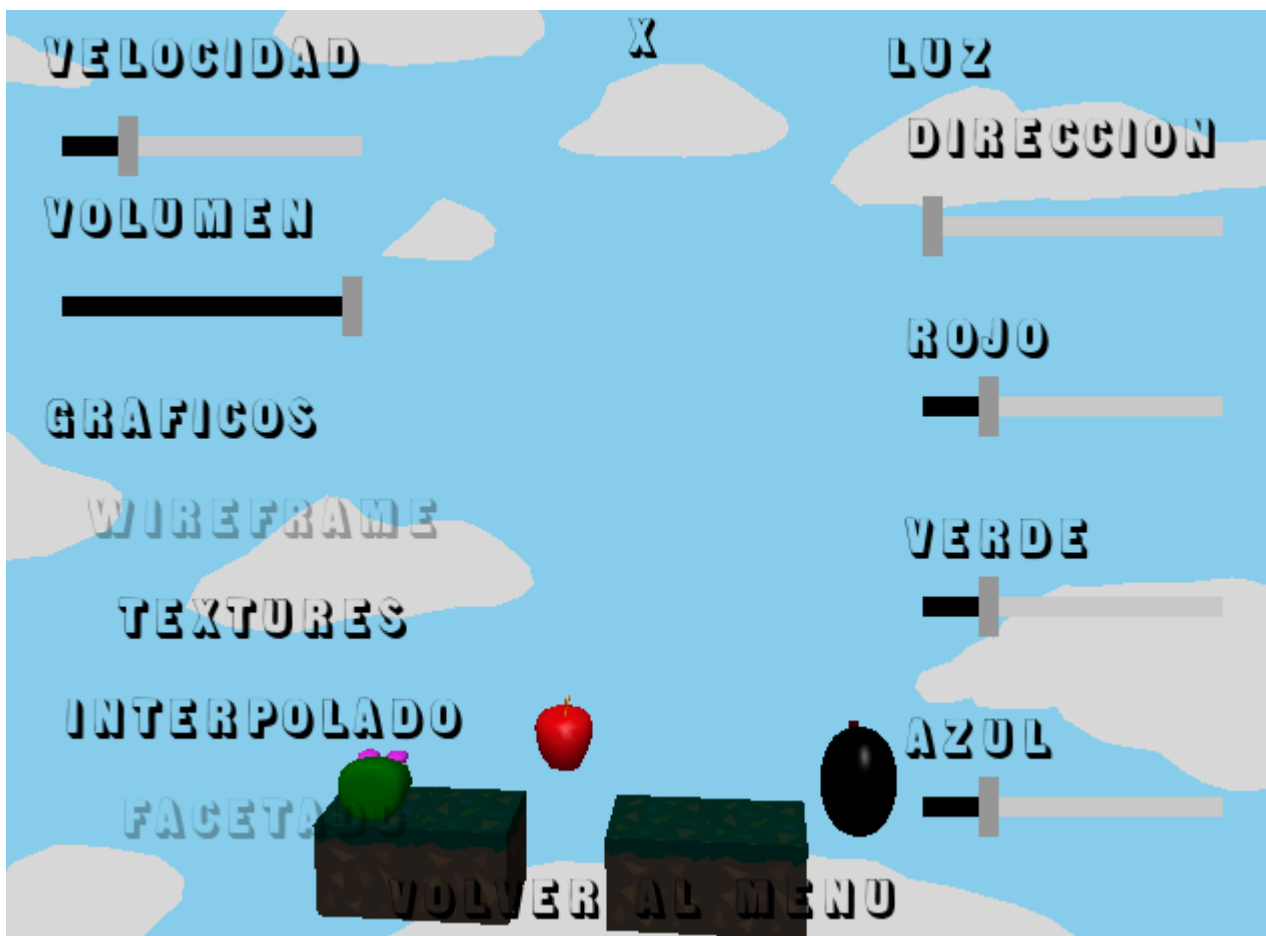
- variables de progreso, estas generalmente van de 0 a 1 y aumentan o decrecen en el process del elemento, luego esta se utiliza en el draw para agregar offsets, rotación o escala. El ejemplo más complejo de estas en nuestra aplicación es el movingProgress de la clase Viborita, que maneja el movimiento y rotaciones fluidas de la viborita y la cámara al hombro.
- variables de ángulo, estas son variables para animaciones cíclicas, siempre se suma una cantidad fija (velocidad de la animación) multiplicada por deltaTime en el process, y en el draw se aplica cos o sin, esto resulta en un valor entre -1 y 1, que permite hacer animaciones sencillas que se repitan. Un ejemplo está en la clase de Apple, que tiene una variable de ángulo que se utiliza para rotar alrededor del eje y, y para que tenga una leve transition en el eje y.

Colisiones

Al desarrollar todo el juego en una grilla es siempre fácil chequear si una posición ya está ocupada y cómo manejarlo.

La única entidad que maneja las colisiones es la Viborita, siempre chequea antes y después de moverse por bloques, manzanas, ella misma, una meta u manzana con pinchos y reacciona acordeamente (morir, crecer, ganar el nivel, caerse, no moverse).

Settings



La interfaz de opciones en el contexto de nuestra arquitectura es simplemente un vector de `IHudElements`, la clase encargada de generar esta lista y hacer que llegue al `HudController` es `SettingsMenu`, si vemos en el UML esta tiene métodos que actualizan los valores de cada configuración, y a su vez tiene un puntero al objeto de `Settings` de la sesión de juego.

Los métodos se llaman por medio de callbacks en los distintos botones.

El `SettingsMenu` es un `IGameState`, por lo que mientras que las opciones están abiertas, este es el estado del juego, sin embargo, por medio del `GameController` accedemos al `GamePlay` actual, y llamamos a su método `draw`, esto nos permite ver el juego mientras estamos en el menú de pausa.

El renderizado de los botones y sliders se hace en la clase `HudController`, ésta se utiliza para todas las pantallas del juego, pero veamos un poco en profundidad cómo funciona para entender el funcionamiento de las opciones.

El `HudController` tiene un método `process(float deltaTime)` este se encarga de:

1. Obtener la posición del mouse en este frame y crear un rectángulo de 1x1 en esta
2. Obtener del `GameController` qué elementos del hud hay que dibujar, los recibe como un vector de punteros a `IHudElement`.
3. Cambia la matriz de proyección para que utilice una perspectiva ortogonal que abarque las dimensiones de la ventana.

-
4. Recorre los IHudElements chequeando para cada uno si el mouse está dentro de su rectángulo, luego con un switch de acuerdo al tipo del elemento del hud realiza el procesamiento necesario, en el caso del menú de opciones se utilizan los siguientes:
 - a. Para botones llama a handleHover en caso de que haya comenzado el hover, o terminado. También llama a handleClick en caso de que haya habido un click mientras está en hover.
 - b. Para los sliders chequea si el mouse está sobre el selector o sobre la barra de progreso, si el mouse está sobre alguno de estos chequea si el mouse está apretado, en cuyo caso llama a mouseDown del slider, si el mouse está arriba llama a mouseUp, finalmente llama al procesMousePos, en el que se actualizará el valor del slider de acuerdo a la posición actual del mouse
 5. Finalmente devuelve la perspectiva a como estaba antes.

Características particulares de la solución

Sonidos

Implementamos algunos sonidos utilizando la librería SDL_mixer [6] para que el juego se sintiera mejor estos fueron:

1. Hover en los botones
2. Comer una manzana
3. Moverse por el nivel

Los sonidos los obtuvimos de Freesound [7]

Sistema de partículas

Implementamos un sistema de partículas sencillo para ciertos efectos en el juego, estos son:

1. Al comer una manzana
2. Al comerse una parte del cuerpo de la viborita
3. El portal

No tuvimos tiempo de agregar todos los efectos de partículas que nos hubiera gustado, como por ejemplo partículas si se choca contra algo la viborita.

Diseño de niveles

Al elegir una jugabilidad distinta tuvimos que cambiar completamente el diseño de niveles con respecto al original, esto es debido a que la libertad del movimiento 3d es demasiado grande, y los desafíos planteados en el juego original no serían tan difíciles.

Pero gracias a esta libertad pudimos diseñar niveles basándonos en otros aspectos de la jugabilidad únicos de nuestro juego, algunas consideraciones al diseñar niveles son:

1. Mayor libertad a la hora de moverse por el nivel, esto significa que los niveles no siempre tienen una solución única, o un orden único en el que comer las manzanas.
2. Orientación de la cámara, esto nos permite esconder elementos y que el jugador tenga que explorar el nivel con la cámara para encontrar los objetivos.
3. Trampas, colocamos varias manzanas “trampas”, esto por ejemplo puede ser que si se come en un orden incorrecto el jugador no se puede recuperar y pierde, puede ser que si se come ya no se puede usar como plataforma para llegar a otras partes del nivel, etc.
4. Progresión, intentamos que los niveles al principio tengan una curva relativamente lineal en los primeros 12 niveles, estos sirven como tutorial (incluyendo gráficos de ayuda) y de ahí en adelante la dificultad va a ir variando con algunos niveles más difíciles que otros.

Algunos desafíos superados

- Cargado de modelos

Al momento de cargar modelos nos encontramos con varios problemas, estos fueron algunos de ellos:

1. El uso de la estructura mesh, esta contiene mucha información, incluyendo las posiciones de los vértices, los índices, las normales, el material (del cual usamos el color de difusión). En el correr del desarrollo fuimos agregando todas estas propiedades a medida que nos dimos cuenta que cosas no funcionaban correctamente:
 - a. Primero solo se cargaba un mesh del modelo, ahí nos dimos cuenta que nuestra implementación estaba usando un mesh solo para todos los vértices, la solución fue utilizar un vector de meshes, que se recorre al momento de dibujar
 - b. Luego al agregar iluminación nos dimos cuenta que no se cargaban las normales de cada vector, por lo que agregamos a nuestro struct de mesh las normales.

-
- c. Finalmente no se estaban respetando los colores de cada mesh que veíamos en blender, ahí descubrimos que teníamos que usar el material para cargar el color de cada mesh.
 2. El posicionamiento y rotación de un modelo. Al principio exportamos de blender los modelos sin considerar la orientación ni posición, esto causaba que tuviéramos que aplicar traslaciones y rotaciones diferentes para cada modelo, esto fue una solución fácil, exportar los modelos centrados en (0,0,0) y adaptando la rotación a los ejes de nuestro juego
 3. Z-fighting, esta fue una crisis de último momento, probando en varias computadoras notamos que los modelos no se veían bien en computadoras de gama más baja que hiciesen uso de gráficos integrados, esto hacía que el cálculo de la profundidad se hiciera con menos precisión y qué parte del mesh va sobre otro no se calcule correctamente, causando z-fighting. La solución fue descubrir sobre la el tamaño del depth_buffer, y que podemos setearla usando la función: `SDL_GL_SetAttribute(SDL_GL_DEPTH_SIZE, 24)`; ya que por defecto, en máquinas con gráficos integrados el tamaño es de solo 16 bits.

- Iluminación

La iluminación no fue implementada desde un principio, lo que causó que se necesitaran varios cambios retroactivos para adaptar los modelos anteriores y que la iluminación se vea correctamente.

Principalmente tuvimos que cambiar cómo dibujamos los cubos, pasando de un array de 24 floats con los vértices del cubo, en este array los vértices se reutilizan varias veces entre las caras, a un array de 72 floats, en el que cada vértice se utiliza una vez. La ventaja de esto es que nos permite definir vector normal a cada vértice individual.

Además como mencionamos anteriormente, tuvimos que actualizar nuestra carga y renderizado de modelos para que utilizara las normales.

- Creación de elementos de la interfaz

Al no usar ninguna librería tuvimos que implementar el funcionamiento de componentes de UI básicos, como un botón, textfield o un slider.

Esto lleva a que nuestras implementaciones no sean las mejores, o estén bastante limitadas. Algunas particularidades de nuestros elementos de UI son:

1. La clase Buttons tiene unas 5 formas de utilizarse distintas:
 - a. Botón con imagen
 - b. Botón con imagen e imagen de hover
 - c. Botón de texto con hover
 - d. Renderizado de texto
 - e. Botón de texto seleccionable
2. La clase textfield no permite seleccionar la posición en la que queremos escribir, solo podemos agregar al final y borrar desde el final. Decidimos dejar solo el funcionamiento básico de escribir y borrar para reducir el tiempo de desarrollo en el componente.
3. La clase LevelButton necesita renderizar cosas en 3d, esto es un problema ya que al momento de llamar a su función de draw, la perspectiva está en modo ortogonal de dos dimensiones, para esto agregamos dos funciones implementadas en IHudElement: `goBackTo3d()` y `goBackTo2d()`

- Diseño de la viborita

El problema de en qué partes separar el modelo de la viborita para poder hacerlo dinámico fue uno que tuvo varias iteraciones, a lo que llegamos al final es a tener 3 modelos, uno con la cabeza, uno con el torso, y uno con la cola. Estos modelos no están unidos y siempre tienen un espacio entre sí, esto es una decisión tanto para reducir complejidad (mantener los modelos unidos

en todas las rotaciones) y de jugabilidad, ya que al ver las partes separadas se puede identificar fácilmente el tamaño de la viborita y su lugar con respecto a la grilla.

- Animaciones de la viborita

Esta fue la funcionalidad que más tiempo llevó desarrollar del juego, animar a la Viborita conlleva varios desafíos no obvios en primera instancia:

1. Recorrer el cuerpo de la viborita
2. Elegir qué modelo renderizar
3. Determinar la rotación, en caso de los extremos estas rotaciones son de 90° , lo cual no parece ser un desafío muy grande, pero en el caso del cuerpo, teníamos que además considerar si es una esquina, por lo que hay que rotarla 45° para unir las otras dos partes.
4. Otro problema particular de las partes del medio, es que si es una esquina, solo rotarla esos 45° no se ve bien, por lo que hay que aplicarle un offset en la dirección hacia donde rota, para que no quede simplemente separado del resto del cuerpo

Esto solo incluye como renderizar correctamente a la Viborita cuando está quieta, animarla trae consigo otras dificultades, ya que no solo hay que trasladarla de una casilla a otra de forma continua, sino que hay que rotar sus partes de forma continua también, es aquí que queda un pequeño error en el juego, en el que al rotar en ciertos ángulos la animación rota el cuerpo 270° en lugar de 90° , esto si bien no es 100% correcto decidimos dejarlo ya que le da cierta personalidad al movimiento.

Conclusiones

Con el desarrollo de este videojuego usando las características de OpenGL, podemos decir que OpenGL nos brinda un gran conjunto de herramientas necesarias para la creación de gráficos 3D, gracias a esto logramos un diseño e implementación satisfactorio de los requerimientos obligatorios de la tarea.

A su vez, esta libertad que nos dan las herramientas también significa que no trae soluciones a problemas de más alto nivel, como por ejemplo renderizar texto, o cargar modelos tridimensionales o elementos para la interfaz. En otros contextos tomaríamos por sentado que todo esto vendría implementado, sin embargo para este juego tuvieron que ser implementadas por nosotros.

Con el avance del proyecto también comenzamos a notar las limitaciones que trae OpenGL, principalmente con la iluminación, que no toma en cuenta sombras, esto podría ser implementado con shaders, sin embargo eso estaba fuera del alcance de esta tarea.

Trabajo futuro

El juego en cuanto a funcionalidades tiene todo lo que querríamos implementar. Sin embargo, por la forma en la que se inició el proyecto acarreamos algunas limitaciones que podríamos mejorar, y siempre se puede agregar más contenido, algunas de las mejoras a futuro que se podrían agregar al juego:

- Usar un map para la grilla, actualmente al usar una matriz de 8x8x8, estamos limitados a ese cubo en el mundo, el cual es bastante reducido, si utilizáramos un map con una clave que represente los índices (por ejemplo una clave string de la forma: "x|y|z"), podríamos tener niveles sin límite de tamaño en ninguna dirección, y no necesariamente aumentar tanto el uso de memoria, ya que usando la matriz siempre la mayoría de las posiciones estarán vacías
- Mejorar la implementación de la cámara en primera persona, actualmente este es el modo menos cómodo de jugar, e incluso intentando diseñar niveles alrededor de la mecánica es muy difícil de aprovechar, ya que la cámara fácilmente se mete en objetos del nivel o no permite ver correctamente lo que hay alrededor. Agregar algún chequeo para acercar la cámara en caso de que haya algo tapando la visibilidad, y aumentar el campo de visión serían mejoras que permitirían un mejor aprovechamiento de la mecánica.
- Agregar más elementos para los niveles, actualmente solo tenemos unas entidades básicas que incluir en los niveles, algunas ideas de otras entidades podríamos agregar:
 - Un bloque empujable, que permita llenar huecos o usar para subir paredes demasiado altas.
 - Un bloque transparente, que permita limitar el movimiento sin necesariamente tener que poner un bloque de tierra que impida la visibilidad.
 - Un bloque con iluminación, esto podría hacer niveles que jueguen con la iluminación para esconder parte del nivel, sin embargo con la implementación actual de iluminación que tenemos esto no sería posible, se necesitaría el uso de shaders para poder generar sombras y efectos más avanzados de iluminación
 - Power ups, por ejemplo, que al comer una pera la viborita pueda comer manzanas con pinchos por cierta cantidad de movimientos.
 - Algún tipo de enemigo, podría imitar tus movimientos y comerse tus manzanas, obstruir tu camino, etc.
- Mejor progresión de niveles, separar los niveles por mundos, que cada mundo tenga una paleta de colores e iluminación propia.

Por otro lado, a nivel de código, hay varias partes que podrían ser mejoradas: mal uso de memoria dinámica, círculos de llamadas a funciones redundantes, funciones que podrían ser operators de los structs, e incluso structs que ya estaban implementados en glm que implementamos nosotros por no conocerlos.

Una revisión para refactorizar esas partes del código podría ser beneficioso para la claridad y performance del mismo.

Referencias

- [1] Modelo de nubes y mapa alrededor del juego:
<https://sketchfab.com/3d-models/star-wars-low-poly-hoth-skybox-caddf0ec811a47d5a2449a4d076a9d7b>
- [2] Documentación de SDL2, algunas páginas relevantes:
<https://learn.microsoft.com/en-us/windows/win32/opengl/gllightfv>
<https://learn.microsoft.com/en-us/windows/win32/opengl/glmaterialfv>
- [3] Documentación de pugixml
<https://pugixml.org/docs/quickstart.html>
- [4] Sistema de partículas
<https://www.youtube.com/watch?v=s8LGqotLc-g>
- [5] Uso de deltaTime
<https://www.youtube.com/watch?v=yGhfUcPjXuE&t>
- [6] SDL_mixer
https://github.com/libsdl-org/SDL_mixer
- [7] Freesound
<https://freesound.org>
- [8] Assimp
<https://github.com/assimp>